

Software Engineering

Chapter 7: Software Testing & Quality Verification

Prof. Nien-Lin Hsueh

Department of Information Engineering and Computer Science
Feng Chia University

Chapter 7: Roadmap & Core Curriculum

Part 1: Principles & Black-Box Testing

- **7.1 Testing Foundations & Principles:** Verification vs. Validation, Dijkstra's axiom, 7 testing principles, testing pyramid, testing stages.
- **7.2 Black-Box Testing Techniques:** Boundary Value Analysis formulas ($4n+1$, $6n+1$, 5^n , 7^n), Equivalence Partitioning (Weak vs. Strong), case studies: Triangle, Exam Scores, FCU Swimming Pool, Binary Search, nextDate.
- **7.3 Combinatorial & Behavioral Methods:** Pairwise/All-Pairs testing, Decision Table testing, State Transition and Use Case testing.

Part 2: White-Box Structural Testing & Synthesis

- **7.4 White-Box Structural Testing:** Subsumption hierarchy, benchmark code analysis, Statement, Branch, and Condition Coverage, short-circuit evaluation.
- **7.5 Advanced Criteria & Control Flow:** Multiple Condition Coverage, MC/DC in DO-178B/C, loop path explosion, McCabe's Cyclomatic Complexity.
- **7.6 Testing Synthesis & Anti-Patterns:** The myth of 100% coverage, comprehensive benchmark comparison, recap quiz, and industry best practices.

7.1 Software Testing Foundations & Principles

"Program testing can be used to show the presence of bugs, but never to show their absence!"

— Edsger W. Dijkstra

”

Testing Objectives: Verification vs. Validation (V&V)

- **Verification ("Are we building the product right?"):**
 - Confirms that software artifacts conform to their technical specifications, architecture designs, and coding standards.
 - Typical techniques: static code analysis, inspections, code reviews, and automated unit test suites.
- **Validation ("Are we building the right product?"):**
 - Demonstrates that the executable system satisfies actual customer needs, operational workflows, and business goals.
 - Typical techniques: user acceptance testing (UAT), customer demos, and beta testing.
- **Two Complementary Testing Goals:**
 1. **Validation Testing:** Demonstrates to stakeholders that the system performs its specified functions correctly under normal inputs.
 2. **Defect Testing:** Intentionally submits invalid, boundary, or hostile inputs to break the system and expose latent flaws.

7 Core Software Testing Principles (Part 1)

- **1. Testing Shows Presence of Defects (Dijkstra's Axiom):**

- Testing proves that bugs exist within an application; it can never prove that software is 100% defect-free.

- **2. Exhaustive Testing is Impossible:**

- Testing every possible combination of inputs, memory states, and preconditions is computationally infeasible.
- Engineers must employ risk-based sampling, boundary analysis, and equivalence partitioning.

- **3. Early Testing (The "Shift-Left" Movement):**

- Testing activities must begin as early as requirements engineering and architectural design.
- Catching defects during requirements is 10 to 100 times cheaper than fixing them in post-deployment production.

- **4. Defect Clustering (The Pareto Principle / 80-20 Rule):**

- In most software systems, roughly 80% of all discovered bugs originate from 20% of the core modules.
- Modules with high cyclomatic complexity, frequent churn, or tight coupling harbor the vast majority of flaws.

7 Core Software Testing Principles (Part 2)

- **5. Beware of the Pesticide Paradox:**

- Running the exact same automated test suite repeatedly will eventually stop finding new defects.
- Just as agricultural pests develop resistance to pesticides, software defects evade repetitive tests; test suites must continually evolve, mutate, and expand.

- **6. Testing is Context Dependent:**

- No single testing strategy fits all projects.
- An e-commerce mobile application emphasizes rapid deployment, usability, and checkout funnel speed; a medical pacemaker or avionics flight controller demands rigorous formal proofs and MC/DC certification.

- **7. Absence-of-Errors Fallacy:**

- Achieving zero reported defects is meaningless if the system is unusable, performs poorly, or solves the wrong customer problem.
- Usability, fitness for purpose, and business alignment override purely technical bug counts.

Visual Overview: 7 Core Software Testing Principles

7 Core Principles of Software Testing (ISTQB)

1. Presence of Defects

Shows bugs exist;
cannot prove 100% bug-free.

2. Exhaustive Impossible

Testing all inputs
is computationally infeasible.

3. Early Testing / Shift-Left

Start early in SDLC
to cut cost up to 100x.

4. Defect Clustering

80% of defects in
20% of modules (Pareto).

5. Pesticide Paradox

Same tests stop catching
new bugs; update suite.

6. Context Dependent

E-commerce vs. Medical
require distinct strategies.

7. Absence of Errors Fallacy

Zero bugs is useless
if system misses user needs.

The Spectrum of Software Testing Types

Spectrum of Software Testing Types



Specialized Testing 1: Regression Testing

- **Definition:**

- Re-executing existing automated test suites against modified code bases to verify that recent code changes, refactorings, or bug fixes have not broken previously functioning behavior.

- **The Regression Risk:**

- Studies show that 20% to 50% of all bug fixes inadvertently introduce new, collateral defects elsewhere in the codebase.

- **Regression in Modern CI/CD Pipelines:**

- Fast unit regression suites run automatically on every `git push` and Pull Request.
- Comprehensive end-to-end regression suites run nightly or prior to scheduled release cutoffs.

- **Test Suite Selection Strategies:**

- **Retest All:** Running every test (comprehensive but slow for large suites).
- **Regression Test Selection (RTS):** Utilizing code coverage maps to run only tests exercising modified methods.

Specialized Testing 2: Fuzz Testing (Fuzzing)

- **Definition:**

- An automated testing technique that generates and injects massive streams of **invalid, malformed, semi-structured, or pseudorandom inputs** into software interfaces.

- **Target Vulnerabilities:**

- Memory corruption (buffer overflows, use-after-free, double-free in C/C++/Rust).
- Unhandled exceptions, infinite loops, thread deadlocks, and denial-of-service crashes.

- **Fuzzing Methodologies:**

- **Mutation-based:** Takes valid seed inputs and mutates bits/bytes randomly.
- **Generation-based:** Generates inputs conforming to a specified grammar or protocol.
- **Coverage-guided:** Uses lightweight binary instrumentation to prioritize inputs that reach new code branches (e.g., AFL, libFuzzer, Google OSS-Fuzz).

Specialized Testing 3: Smoke, Sanity & Mutation Testing

- **Smoke Testing (Build Verification):**

- A fast, non-exhaustive test suite executed on every fresh software build to verify core critical paths ("Does the build catch fire when turned on?").
- If smoke tests fail, the build is immediately rejected and returned to developers without wasting QA team time.

- **Sanity Testing (Post-Fix Verification):**

- A focused, narrow test suite executed immediately following a minor bug fix or patch to confirm that the specific reported defect is resolved before broader regression.

- **Mutation Testing (Evaluating the Test Suite Itself):**

- Intentionally injects small syntactic faults ("mutants", e.g., swapping `>` with `<`, `+` with `-`) into the source code.
- If existing test suites fail ("kill the mutant"), the test suite is strong; if tests pass despite the corrupted code ("surviving mutant"), test coverage is inadequate.

Non-Functional Testing: Load, Stress & Security

- **Load Testing:**

- Validates system response time, throughput (RPS), and server resource consumption under **expected peak operational workloads** (e.g., 5,000 concurrent shopping users).

- **Stress Testing:**

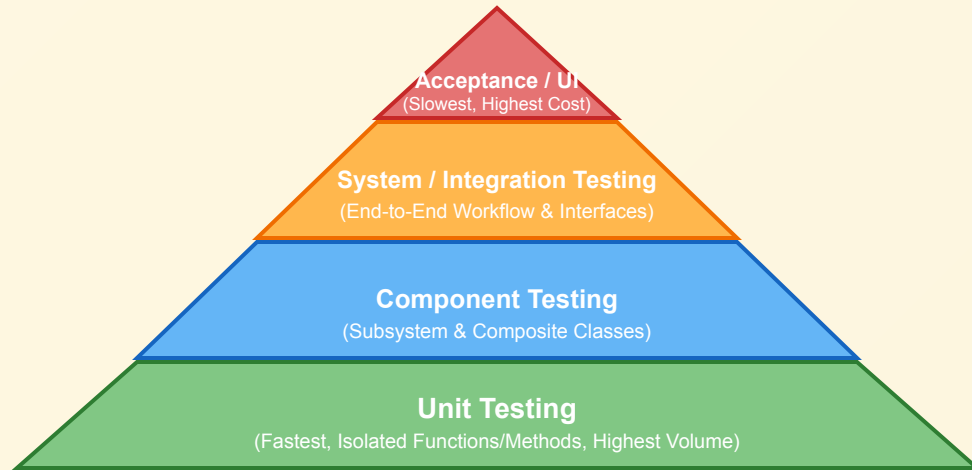
- Pushes system concurrency and transaction volume **beyond maximum design limits** until failure occurs.
- Objective: Evaluates graceful degradation, error handling, and automated recovery without data corruption.

- **Security & Penetration Testing:**

- Validates defense-in-depth against malicious exploits: SQL injection, cross-site scripting (XSS), broken access control, and unauthorized privilege escalation.
- Combines automated Static/Dynamic Application Security Testing (SAST/DAST) with ethical manual red-teaming.

The Software Testing Pyramid

The Software Testing Pyramid



Stages of Testing: Development Testing

- **Unit Testing:**

- Evaluates individual functions, algorithms, or object classes in strict isolation.
- Isolates external dependencies (databases, external APIs, filesystem) using **test doubles (mocks, stubs, and fakes)**.
- Executed locally by developers and in automated pre-commit hooks.

- **Component Testing:**

- Evaluates clusters of cooperating classes that together form a cohesive functional component or subsystem.
- Focuses on module boundary interfaces and internal data flow integrity.

- **System Testing:**

- Integrates all subsystems into a complete runtime package to verify end-to-end workflows and emergent behavior.
- Verifies cross-service communication, security policies, and performance characteristics in staging environments.

Release Testing vs. User Testing

- **Release Testing (Pre-Deployment Validation):**
 - Performed by an independent QA team on a candidate release build.
 - **Requirements-Based Testing:** Systematically checks off every acceptance criterion in the functional specification.
 - **Scenario / Workflow Testing:** Simulates realistic, end-to-end user journeys (e.g., browsing, adding to cart, checkout, delivery tracking).
- **User & Acceptance Testing (Customer Evaluation):**
 - **Alpha Testing:** Internal employees and trusted simulated users test the software on-site at the developer's facility.
 - **Beta Testing:** Early release delivered to external end-users in real operating environments to gather field feedback and telemetry.
 - **Acceptance Testing:** The formal verification performed by the client against legal contract requirements prior to commercial sign-off.

Concept Check Question 1: Testing Principles

According to Dijkstra's foundational testing axiom and the Shift-Left principle, what is the primary capability and economic timing of software testing?

- **A.** It mathematically proves that a program is completely free of all defects when executed early in requirements.
- **B.** It can show the presence of defects but never their absence, and finding bugs early is drastically cheaper.
- **C.** It guarantees 100% statement coverage across all modules if automated regression tests are run continuously.
- **D.** It replaces the necessity for customer acceptance testing by eliminating defect clustering in production code.

Concept Check Question 1: Answer & Explanation

- **Correct Answer: B**

- **Explanation:**

- Edsger Dijkstra's famous axiom explicitly states: "*Program testing can be used to show the presence of bugs, but never to show their absence!*" Testing provides empirical evidence of flaws, but cannot mathematically prove zero bugs exist.
- The Shift-Left principle emphasizes testing as early as requirements and design, because resolving defects early costs 10x to 100x less than fixing them in post-release maintenance.
- *Why others are incorrect:* Option A contradicts Dijkstra's axiom. Option C confuses structural statement coverage with functional defect absence. Option D is false because automated tests never eliminate the need for customer acceptance testing.

7.2 Black-Box Testing Techniques

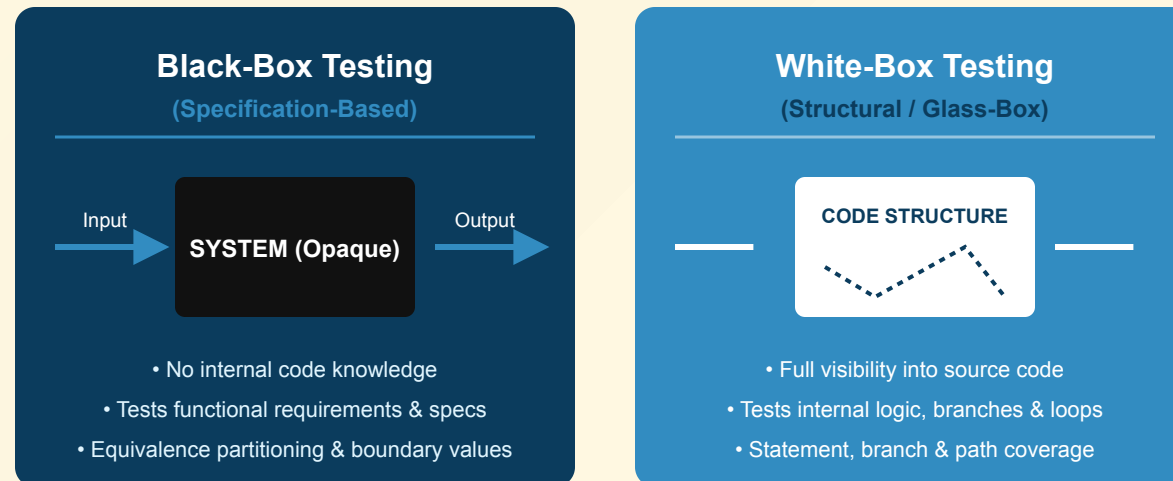
"Testing without a specification is like exploring a dark cave without a map: you might find something, but you have no idea if you're in the right cavern."

”

Black-Box vs. White-Box Testing

- **Black-Box Testing (Specification-Based / Functional):**
 - **Perspective:** The tester treats the application as an opaque black box with zero visibility into source code, algorithms, or internal data structures.
 - **Input Source:** Derived directly from functional requirements, user stories, API contracts, and user documentation.
 - **Primary Benefit:** Tests whether the system fulfills intended user goals; completely unbiased by developer implementation quirks.
- **White-Box Testing (Structural / Glass-Box / Clear-Box):**
 - **Perspective:** The tester has complete visibility into the source code, control flow graphs, conditions, and execution branches.
 - **Input Source:** Derived from control logic, loops, conditional statements, and architectural structures.
 - **Primary Benefit:** Identifies dead code, unexercised logic branches, and boundary edge cases inside algorithms.

Visual Overview: Black-Box vs. White-Box Testing



5 Major Black-Box Testing Methods

5 Key Black-Box Testing Techniques

1. Equivalence Partitioning

Divides input domain into valid & invalid equivalence classes.

2. Boundary Value Analysis

Tests values at partition edges (min-1, min, max, max+1).

3. Decision Table Testing

Maps combinations of complex logical conditions to actions.

4. State Transition Testing

Verifies system responses across discrete finite state transitions.

5. Use Case / Scenario

Derives test cases directly from end-to-end user workflows.

Boundary Value Testing: The Single Fault Assumption

- **Why Boundaries Fail (The "Off-by-One" Phenomenon):**
 - Programmers frequently make comparison mistakes (`<` vs. `<=`, `>` vs. `>=`) and index errors at partition limits.
 - Empirical research confirms that **over 60% of functional bugs occur at boundaries** of input domains.
- **The Single Fault Assumption (單一錯誤假設):**
 - Assumes that software failures are almost always caused by a defect in a **single variable** rather than simultaneous, coordinated defects across multiple variables.
 - *Consequence:* When testing the boundary values of variable X , all other variables are held at their typical, nominal middle values (`norm`).
- **Worst-Case Testing (Non-Independent Variables):**
 - Rejects the single fault assumption when input variables interact in compound logical expressions (e.g., `if (exam <= 60 && hw <= 60)`).
 - Generates the complete **Cartesian cross-product** of boundary values across all inputs.

The 4 Boundary Value Testing Formulas

Black-Box Boundary Value Testing Taxonomy

Testing Type / Assumption	Normal Boundary (min, max)	Robust Boundary (min-, max+)
Independent Testing (Single Fault Assumption)	$4n + 1$ min, min+, norm, max-, max	$6n + 1$ Adds out-of-bounds (min-, max+)
Non-Independent (Worst-Case / Cartesian Product)	5^n Combinatorial cross-product ($5^3 = 125$)	7^n Robust cross-product ($7^3 = 343$)

- For n input variables, the required test cases are:

- Independent Normal (BVA):** $4n + 1$ (values: min, min+, norm, max-, max per variable, plus 1 all-norm).
- Independent Robustness:** $6n + 1$ (adds out-of-bounds min- and max+).
- Worst-Case Normal:** 5^n (Cartesian product of 5 points across all variables).
- Worst-Case Robustness:** 7^n (Cartesian product of 7 points across all variables).

Boundary Case Study: Triangle Classification

- **Problem Specification:**

- Inputs: Three integers $a, b, c \in [1, 200]$ representing side lengths.
- Output: *Equilateral*, *Isosceles*, *Scalene*, or *Non-Triangle*.

- **Independent Normal Boundary ($4n + 1 = 13$ test cases):**

- Hold $b, c = 100$ (**norm**). Test $a \in \{1, 2, 199, 200\}$.
- Repeat for b (holding $a, c = 100$) and c (holding $a, b = 100$), plus 1 case $(100, 100, 100)$.

- **The "Diversity Deficit" Flaw:**

- Notice that in all 13 test cases, two sides are *always equal to 100*!
- Therefore, the 13 test cases only test *Equilateral* $(100, 100, 100)$ and *Isosceles* triangles;
not a single Scalene triangle is ever tested!

- **Engineering Solutions:**

- Use dynamic random **norm** values for each variable (e.g.,
 $a_{\text{norm}} = 100, b_{\text{norm}} = 101, c_{\text{norm}} = 102$).
- Apply **Output-Guided Partitioning** to guarantee coverage across all expected output categories.

Equivalence Partitioning: Weak vs. Strong Coverage

- **Equivalence Partitioning (EP) Principle:**

- Partitions the infinite input and output space into discrete classes where the program is presumed to process all values within a class identically.
- Testing one representative value from each partition is theoretically equivalent to testing all values in that partition.

- **Weak Coverage (Weak EP):**

- Based on the **Single Fault Assumption**.
- Requires that each identified partition of every variable is covered by **at least one test case**.
- Total test cases = $\max(|P_1|, |P_2|, \dots, |P_n|)$.

- **Strong Coverage (Strong EP):**

- Rejects single fault assumption to account for multi-variable interactions.
- Requires testing the **Cartesian product** of all partitions across all variables:

$$\text{Total Test Cases} = |P_1| \times |P_2| \times \dots \times |P_n|$$

EP Case Study 1: Single-Variable Domain (Exam Scores)

Specification: *An academic grading portal accepts integer scores between 0 and 100 inclusive.*

”

- **Partition 1: Invalid Low ($\text{Score} < 0$):**
 - *Representative Test Value:* -15
 - *Expected System Output:* Rejection error ("Score cannot be negative").
- **Partition 2: Valid Range ($0 \leq \text{Score} \leq 100$):**
 - *Representative Test Value:* 75
 - *Expected System Output:* Score accepted and grade recorded.
- **Partition 3: Invalid High ($\text{Score} > 100$):**
 - *Representative Test Value:* 135
 - *Expected System Output:* Rejection error ("Score exceeds maximum limit of 100").
- **Testing Takeaway:**
 - A comprehensive suite must always include both **valid partitions** (testing feature correctness) and **invalid partitions** (testing defensive exception handling).

EP Case Study 2: Multi-Variable FCU Swimming Pool Fee

Specification: Public pool ticket price depends on three variables: **Age**, **Time Slot**, and **Membership Status**.

Multi-Variable Equivalence Partitioning: Swimming Pool Fee

Partition / Test	Age Partition	Time Slot	Member?	Expected Fee
Case 1 (Child)	Age = 8 (<12)	Off-Peak	Yes (Member)	\$50 (Child Discount)
Case 2 (Adult)	Age = 30 (12-64)	Peak (Evening)	No (Non-Member)	\$150 (Full Price)
Case 3 (Senior)	Age = 70 (>=65)	Off-Peak	No (Non-Member)	\$80 (Senior Rate)
Case 4 (Invalid)	Age = -5 (<0)	Peak	Yes	ERROR: Invalid Age

- **Identified Input Partitions:**

- **Age (P_{Age}):** Child (< 12), Adult (12--64), Senior (≥ 65), Invalid (< 0).
- **Time Slot (P_{Time}):** Off-Peak (Morning), Peak (Evening).
- **Membership (P_{Member}):** Member (Discounted), Non-Member (Standard).

- **Coverage Analysis:**

- **Weak EP:** $\max(4, 2, 2) = 4$ test cases (covers all partitions at least once).

EP Case Study 3: Binary Search Partition Plan

Specification: `Search(Key: int, A: Array) -> (Found: bool, Index: int)`

”

- **Input Array Partitions:**

- Empty array (a^0 : length = 0).
- Single-element array (a^1 : length = 1).
- Multi-element sorted array (a^* : length > 1).

- **Output & Target Position Partitions:**

- **Found (f^t):** Key located at First index (c^1), Middle index (c^m), or Last index (c^l).
- **Not Found (f^f):** Key smaller than array min ($v^{<\min}$), larger than array max ($v^{>\max}$), or between elements.
- **Invalid ($k^!$):** Null or unsorted array input.

- **Weak Coverage Plan:**

- Derive 8 structured test cases (R_1 -- R_8) ensuring every input state and output position is verified.

EP Case Study 3: Binary Search Test Suite

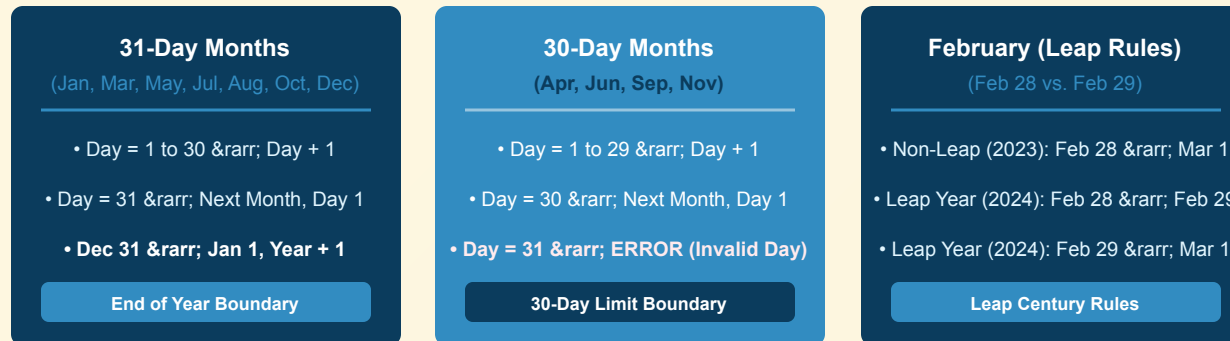
Binary Search EP Weak Coverage Test Suite (8 Test Cases R1 - R8)

Case	Partition Rule Description	Key	Input Array (A)	Expected Output (Found, Index)
R1	1 element, Found	5	[5]	(Found=True, Index=0)
R2	1 element, Not Found	5	[8]	(Found=False, Index=-1)
R3	Multi elements, First Pos (c1)	5	[5, 12, 20]	(Found=True, Index=0)
R4	Multi elements, Mid Pos (cm)	12	[5, 12, 20]	(Found=True, Index=1)
R5	Multi elements, Last Pos (cl)	20	[5, 12, 20]	(Found=True, Index=2)
R6	Multi elements, Not Found	99	[5, 12, 20]	(Found=False, Index=-1)
R7	0 elements (Empty Array)	5	[]	(Found=False, Index=-1)
R8	Invalid Input / Null Key	null	[5, 12, 20]	ERROR: Invalid Input

Boundary & EP Case Study: nextDate Plan

Specification: `nextDate(month: int, day: int, year: int) -> String` where $Year \in [1800, 2048]$.

nextDate(month, day, year) Boundary & Equivalence Classes



• Critical Boundary Test Conditions:

- Standard Month End: `2024-01-31` → `2024-02-01`
- Year End Rollover: `2024-12-31` → `2025-01-01`
- 30-Day Month Boundary: `2024-04-30` → `2024-05-01` (Day 31 in April is Invalid!).
- Leap Year Century Boundary:

Prof. Nien-Lin Hsu `2024-02-28` → `2024-02-29` (Leap year divisible by 4).

Boundary & EP Case Study: nextDate Test Suite

nextDate(month, day, year) Concrete Test Suite (TC1 - TC8)

Case	Boundary / Rule Category	Input Date (m, d, y)	Expected Result / Output
TC1	Normal Day Increment	month=3, day=15, year=2024	2024-03-16
TC2	31-Day Month End	month=1, day=31, year=2024	2024-02-01 (Month Rollover)
TC3	30-Day Month End	month=4, day=30, year=2024	2024-05-01 (April Rollover)
TC4	30-Day Invalid Day	month=4, day=31, year=2024	ERROR: Apr 31 Invalid
TC5	Leap Year Feb 28	month=2, day=28, year=2024	2024-02-29 (Leap Day)
TC6	Non-Leap Year Feb 28	month=2, day=28, year=2023	2023-03-01 (March 1)
TC7	End of Year Rollover	month=12, day=31, year=2024	2025-01-01 (New Year)
TC8	Out-of-Bounds Month	month=13, day=15, year=2024	ERROR: Invalid Month

Concept Check Question 2: Boundary Value Analysis

Under the Single Fault Assumption, how many test cases are required to perform Independent Normal Boundary Value Analysis on a function with $n = 4$ independent input variables?

- **A.** 16 test cases, derived by evaluating the 2^n binary combinations of boundary extremes.
- **B.** 17 test cases, derived using the $4n + 1$ formula covering 4 boundaries per input plus 1 nominal.
- **C.** 25 test cases, derived using the $6n + 1$ formula including robust out-of-bounds values.
- **D.** 625 test cases, derived using the 5^n worst-case Cartesian product of all inputs.

Concept Check Question 2: Answer & Explanation

- **Correct Answer: B**

- **Explanation:**

- For Independent Normal Boundary Value Analysis under the Single Fault Assumption, each variable is tested at 4 boundary values (`min`, `min+`, `max-`, `max`) while all other variables are held at their nominal middle value (`norm`).
- Across n variables, this yields $4 \times n$ test cases. Adding the single common test case where all variables are nominal (`norm`, `norm`, ...) gives the standard formula:

$$\text{Total Test Cases} = 4n + 1 = 4(4) + 1 = 17 \text{ test cases}$$

- *Why others are incorrect:* Option A (16) confuses BVA with binary truth tables. Option C (25) corresponds to Robustness testing ($6n + 1 = 6(4) + 1 = 25$). Option D (625) corresponds to Worst-Case testing ($5^n = 5^4 = 625$).

7.3 Combinatorial & Behavioral Methods

"Combinatorial explosion is the nemesis of exhaustive testing; pairwise testing is the engineer's sharpest scalpel."

”

All-Pairs (Pairwise) Testing

- **The Combinatorial Explosion Problem:**

- Testing all combinations (Strong EP / Cartesian Product) across k parameters each with v values requires v^k **test cases**.
- Example: A web application with 10 dropdowns, each having 3 choices, requires $3^{10} = \mathbf{59,049}$ test cases!

- **The Empirical Pairwise Principle:**

- Studies by the US National Institute of Standards and Technology (NIST) show that **between 67% and 93% of all software defects are triggered by interactions between at most 2 parameters (pairs)**.

Combinatorial Explosion vs. All-Pairs (Pairwise) Testing

Full Combinatorial (Cartesian)

Tests all possible combinations across all N variables ($N_1 \times N_2 \times \dots \times N_k$).

Exponential Explosion: $O(N^k)$

e.g., 10 variables w/ 3 values = 59,049 tests!

VS

All-Pairs (Pairwise Testing)

Ensures EVERY PAIR of variables is tested together at least once.

Drastic Reduction: $O(N^2)$

e.g., 59,049 tests reduced to ~15-20 tests!

Decision Table Testing: Modeling Complex Boolean Logic

- **Concept & Objective:**

- A structured tabular method for specifying and testing complex business rules where multiple input conditions determine specific system actions.

- **Decision Table Structure:**

- **Condition Stubs:** Lists all boolean conditions/inputs.
- **Action Stubs:** Lists all possible system actions/outputs.
- **Rules (Columns):** Each column represents a unique combination of conditions (T/F) and the resulting triggered actions (X).

Decision Table Example: E-Commerce Discount Rules

Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4
Is VIP Member?	T	T	F	F
Order Total > \$100?	T	F	T	F
Action: Apply Discount	20% Off	10% Off	5% Off	0% (None)

State Transition & Use Case Scenario Testing

- **State Transition Testing:**

- Used for reactive, stateful systems (e.g., e-commerce orders, embedded controllers, game engines).
- Models system states, event triggers, guards, and output transitions.
- **Coverage Goals:**
 1. *All-States Coverage*: Every valid state visited at least once.
 2. *All-Transitions Coverage*: Every valid state transition executed.
 3. *Invalid Transition Testing*: Verifying that invalid transitions are rejected (e.g., trying to *Ship* an order that is *Cancelled*).

- **Use Case & Scenario Testing:**

- Derived directly from user stories and UML Use Case specifications.
- **Happy Path (Basic Flow)**: Tests the primary normal sequence achieving user goals.
- **Alternate & Exception Flows**: Tests validation failures, network disconnects, and user cancellations.

Concept Check Question 3: Combinatorial Testing

Why is All-Pairs (Pairwise) testing widely considered one of the highest-ROI test design techniques in modern software engineering?

- **A.** It tests all possible N -way parameter combinations, guaranteeing 100% path coverage.
- **B.** It requires full white-box access to control flow graphs, eliminating the need for integration testing.
- **C.** It dramatically compresses test suites while empirically catching over 70% to 90% of parameter interaction bugs.
- **D.** It replaces boundary value analysis by automatically generating all invalid boundary points.

Concept Check Question 3: Answer & Explanation

- **Correct Answer: C**

- **Explanation:**

- NIST empirical studies across diverse software domains demonstrate that the vast majority of software defects (67% to 93%) are caused by interactions between at most 2 parameters (pairs).
- All-Pairs testing drastically compresses test suites from $O(v^k)$ down to $O(v^2 \log k)$ (e.g., reducing tens of thousands of tests to under 25) while detecting the overwhelming majority of interaction defects, delivering extraordinary return on investment.
- *Why others are incorrect:* Option A is false because All-Pairs only tests 2-way interactions, not all N -way combinations. Option B is false because All-Pairs is a black-box technique requiring no source code access. Option D is false because Pairwise testing is complementary to Boundary Value Analysis, not a replacement for it.

7.4 White-Box Structural Testing

"You cannot inspect quality into a product, but structural testing ensures no dark corners remain uninspected."

”

White-Box Fundamentals & Coverage Subsumption Hierarchy

- **Core Definition:**

- Testing software with full knowledge of internal source code, control flow graphs (CFG), and logic expressions.

- **The Subsumption Principle ($A \implies B$):**

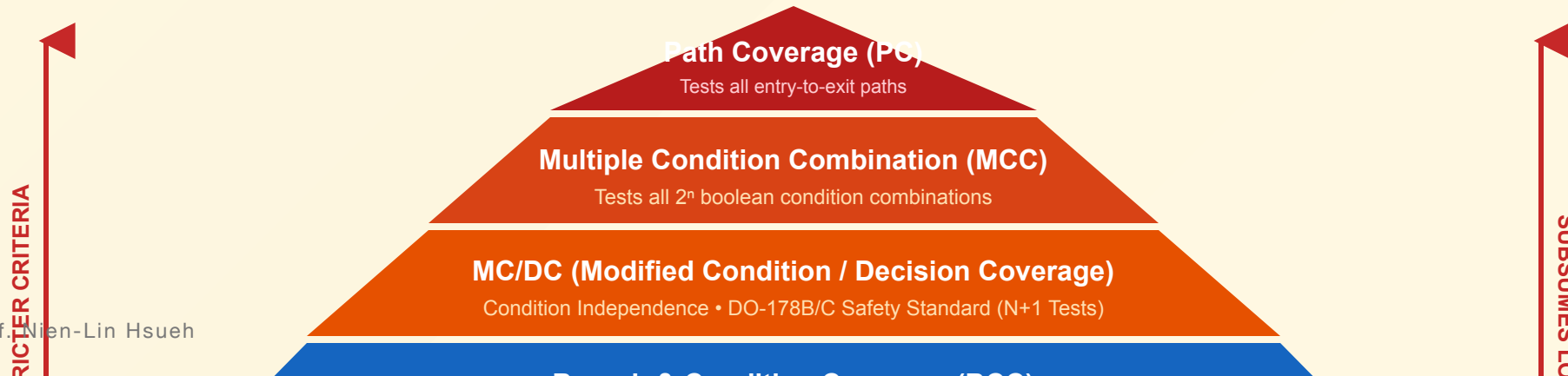
- Coverage criterion A **subsumes** criterion B if and only if any test suite that satisfies A is guaranteed to satisfy B .

- **The Classical Subsumption Hierarchy:**

- Statement (SC) $\leq$ Branch (BC) $\leq$ Branch/Condition (BCC) $\leq$ MC/DC $\leq$ Multiple Condition (MCC) $\leq$ Path

White-Box Code Coverage Subsumption Hierarchy

Higher levels subsume (guarantee) all lower coverage criteria below them



Standard Benchmark Program for Coverage Analysis

To compare all white-box coverage criteria rigorously, we evaluate the **SAME standard benchmark program** across all criteria:

```
// Standard Benchmark Program for Structural Coverage Analysis
void process(int A, int B, int X) {
    if (A > 1 && B == 0) { // Decision 1 (b1): Condition c1 && Condition c2
        Y = A;
    }
    if (A == 2 || X > 1) { // Decision 2 (b2): Condition c3 || Condition c4
        Y = X;
    }
    print(Y);
}
```

- **Decision 1 (b_1):** $(A > 1) \ \&\& \ (B == 0)$
 - Atomic conditions: $c_1 : (A > 1)$, $c_2 : (B == 0)$.
- **Decision 2 (b_2):** $(A == 2) \ || \ (X > 1)$
 - Atomic conditions: $c_3 : (A == 2)$, $c_4 : (X > 1)$.

1. Statement Coverage (SC / SC100)

- **Definition:**

- Every executable statement in the program must be executed **at least once**.
- Formula: Statement Coverage = $\frac{\text{Executed Statements}}{\text{Total Executable Statements}} \times 100\%$.

- **100% Statement Coverage on Benchmark Code:**

- Consider a single test case: $T_1 = (A = 2, B = 0, X = 3)$.
- *Trace:* b_1 evaluates to True $\implies Y = 2$. Next, b_2 evaluates to True $\implies Y = 3$. Then `print(3)` executes.
- **Result: 100% Statement Coverage is achieved with only ONE test case!**

- **The Critical Weakness of Statement Coverage:**

- SC100 is the **weakest** structural criterion!
- If Line 2 contained a bug (`&&` miswritten as `||`) or Line 3 was miswritten as `Y = B`, test T_1 would pass with 100% coverage, completely blind to the defect.

2. Branch / Decision Coverage (BC / BC100)

- **Definition:**

- Every decision point (b_1, b_2) must evaluate to **True** at least once and **False** at least once.
- Formula: Branch Coverage = $\frac{\text{Executed Decision Outcomes}}{\text{Total Decision Outcomes}} \times 100\%$.

- **100% Branch Coverage on Benchmark Code (2 Test Cases):**

- $T_1 = (A = 3, B = 0, X = 3) \implies b_1 = \text{True}, b_2 = \text{True}$
- $T_2 = (A = 3, B = 1, X = 1) \implies b_1 = \text{False}, b_2 = \text{False}$
- Both decisions b_1 and b_2 evaluate to $\{\text{True}, \text{False}\}$.

- **Subsumption Rule ($BC \implies SC$):**

- Achieving 100% Branch Coverage **strictly guarantees 100% Statement Coverage** ($BC100 \implies SC100$).
- For standard commercial software, Branch Coverage is considered the standard engineering baseline.

3. Condition Coverage (CC / CC100) & The Paradox

- **Definition:**

- Every individual atomic boolean condition (c_1, c_2, c_3, c_4) within decisions must evaluate to **True** and **False** at least once.

- **100% Condition Coverage on Benchmark Code (2 Test Cases):**

- $T_1 = (2, 0, 3) \implies c_1 : T, c_2 : T, c_3 : T, c_4 : T$
- $T_2 = (1, 1, 1) \implies c_1 : F, c_2 : F, c_3 : F, c_4 : F$
- All atomic conditions take both True and False values.

- **The CC vs. BC Paradox: Does CC100 guarantee BC100? NO!**

- Consider test suite: $T_a = (3, 1, 1)$ ($c_1 : T, c_2 : F \implies b_1 : \text{False}$) and $T_b = (1, 0, 2)$ ($c_1 : F, c_2 : T \implies b_1 : \text{False}$).
- Both conditions c_1 and c_2 evaluate to $\{T, F\}$ (100% Condition Coverage achieved!).
- **However, b_1 evaluates to False in BOTH tests! 0% Branch True coverage is achieved!**

Short-Circuit Evaluation Impact on Testing

- **Short-Circuit Evaluation Semantics (`&&`, `||`):**

- In modern programming languages (Java, C, C++, JavaScript, Python), right-hand conditions are skipped if the left-hand condition determines the final outcome:

- `false && expr` $\implies$ `expr` is **never evaluated**.
- `true || expr` $\implies$ `expr` is **never evaluated**.

- **Impact on Structural Coverage Measurement:**

Test Case	$p (c_1)$	$q (c_2)$	Short-Circuit Evaluation of $p \ \&\& \ q$
T_1	True	False	Evaluates both; returns False .
T_2	False	<i>Skipped (x)</i>	Skips q ; returns False .
T_3	True	True	Evaluates both; returns True .

- **Key Takeaway:**

- Under short-circuiting, test T_2 never actually exercises condition q .
- To achieve 100% Condition Coverage under short-circuiting compilers, a test like T_3 is required to force $q = \text{True}$, which **simultaneously forces the decision to True and guarantees Branch Coverage!**

Concept Check Question 4: Condition vs. Branch Coverage

Given the decision statement `if (X > 10 && Y == 1) :`

Which test suite achieves **100% Condition Coverage (CC100)** but completely **FAILS** to achieve **Branch Coverage (BC100)**?

- **A.** $T_1 : (X = 12, Y = 1)$ and $T_2 : (X = 5, Y = 2)$
- **B.** $T_1 : (X = 12, Y = 2)$ and $T_2 : (X = 5, Y = 1)$
- **C.** $T_1 : (X = 12, Y = 1)$ and $T_2 : (X = 12, Y = 2)$
- **D.** $T_1 : (X = 5, Y = 1)$ and $T_2 : (X = 5, Y = 2)$

Concept Check Question 4: Answer & Explanation

- **Correct Answer: B**

- **Explanation:**

- Let atomic conditions be $c_1 : (X > 10)$ and $c_2 : (Y == 1)$.
- **In Test Suite B:**
 - For $T_1(12, 2)$: $c_1 = \text{True}, c_2 = \text{False} \implies \text{Decision} = \text{False}$.
 - For $T_2(5, 1)$: $c_1 = \text{False}, c_2 = \text{True} \implies \text{Decision} = \text{False}$.
 - *Condition Coverage*: $c_1 \in \{T, F\}$ and $c_2 \in \{T, F\} \implies$ **100% Condition Coverage achieved!**
 - *Branch Coverage*: Both tests evaluate to False $\implies$ **0% True Branch executed! BC100 fails!**
- *Why others are incorrect*: In Option A, T_1 produces True and T_2 produces False (achieving BC100). In Option C, c_1 is never False. In Option D, c_1 is never True.

7.5 Advanced Criteria & Control Flow

"In safety-critical avionics, an untested condition branch is not an oversight—it is a hazard." „

4. Multiple Condition Combination (MCC)

- **Definition:**

- Every possible combination of boolean truth values for atomic conditions within each decision must be executed.
- For a decision with n atomic conditions, requires 2^n **test cases per decision**.

- **Truth Combinations for Benchmark Code ($2^2 + 2^2 = 8$ combinations):**

- **Decision 1 (c_1, c_2):** (1) TT, (2) TF, (3) FT, (4) FF.
- **Decision 2 (c_3, c_4):** (5) TT, (6) TF, (7) FT, (8) FF.

- **100% MCC Test Suite (4 Test Cases):**

- $T_1 = (2, 0, 4) \implies$ Decision 1: TT $\rightarrow$ True, Decision 2: TT $\rightarrow$ True
- $T_2 = (2, 1, 1) \implies$ Decision 1: TF $\rightarrow$ False, Decision 2: TF $\rightarrow$ True
- $T_3 = (1, 0, 2) \implies$ Decision 1: FT $\rightarrow$ False, Decision 2: FT $\rightarrow$ True
- $T_4 = (1, 1, 1) \implies$ Decision 1: FF $\rightarrow$ False, Decision 2: FF $\rightarrow$ False

- **Limitation:** Exponential explosion: a compound decision with 10 conditions requires $2^{10} = 1,024$ test cases!

5. Modified Condition / Decision Coverage (MC/DC)

- **Safety-Critical Standard:**

- Mandated by **FAA / RTCA DO-178B/C (Level A)** for commercial aircraft flight control software.

- **The 4 MC/DC Requirements:**

1. Every decision point has evaluated to **True** and **False** at least once (BC100).
2. Every condition within a decision has evaluated to **True** and **False** at least once (CC100).
3. Every entry and exit point has been invoked at least once.
4. **Condition Independence:** Each condition has been shown to **independently affect the decision's outcome** by toggling that condition while holding all other conditions fixed.

- **Linear Efficiency ($N + 1$):**

- For a decision with N conditions, MC/DC requires only $N + 1$ **test cases** instead of 2^N !
- For $N = 10$ conditions: MCC requires 1,024 tests; MC/DC requires only **11 test cases**!

MC/DC Independence Pair Analysis & Benchmark Suite

- **Independence Pair Concept:**

- For condition c_i , find two test cases where c_i toggles ($T \leftrightarrow F$), all other conditions c_j remain identical, and the **final decision outcome toggles** ($T \leftrightarrow F$).

- **MC/DC Test Suite for Benchmark Decision 1:** `(A > 1 && B == 0)` :

Test ID	Input (A, B)	$c_1 : (A > 1)$	$c_2 : (B == 0)$	Decision Outcome (b_1)	Validates Independence Of
M_1	(2, 0)	True	True	True	Base True Case
M_2	(1, 0)	False	True	False	Independence Pair for c_1 (M_1, M_2)
M_3	(2, 1)	True	False	False	Independence Pair for c_2 (M_1, M_3)

- **Result:**

- Test suite $\{M_1, M_2, M_3\}$ achieves **100% MC/DC with $N + 1 = 2 + 1 = 3$ test cases!**

6. Path Coverage & The Loop Explosion Problem

- **Definition:**

- Tests **every possible execution path** from program entry to program exit.
- Conceptually the most exhaustive structural criterion possible.

- **Paths for Benchmark Program (4 Independent Paths):**

- Path 1: Decision 1 True, Decision 2 True $\implies (2, 0, 4)$
- Path 2: Decision 1 True, Decision 2 False $\implies (2, 0, 1)$
- Path 3: Decision 1 False, Decision 2 True $\implies (1, 0, 2)$
- Path 4: Decision 1 False, Decision 2 False $\implies (1, 1, 1)$

- **The Loop Path Explosion Problem:**

- If a program contains a loop that executes up to 20 times, and inside the loop there are 4 branching paths:

$$\text{Total Execution Paths} = 4^{20} = 2^{40} \approx 1.1 \times 10^{12} \text{ paths!}$$

- At 1 millisecond per test, executing full path coverage would take **over 34 years!**

Basis Path Testing (McCabe's Cyclomatic Complexity)

- **McCabe's Cyclomatic Complexity $V(G)$:**
 - Measures the structural complexity of a program and defines the maximum number of **linearly independent basis paths** in a Control Flow Graph.
 - **Formulas:**
 1. $V(G) = E - N + 2P$ (where E = edges, N = nodes, P = connected components).
 2. $V(G) = \text{Binary Decision Predicates} + 1$.
 3. $V(G) = \text{Number of Enclosed Regions in Planar CFG} + 1$.
- **Core Engineering Benefit:**
 - Guarantees 100% Statement Coverage and 100% Branch Coverage by executing exactly $V(G)$ **basis paths**, completely bypassing loop path explosion.
- **Complexity Guidelines:**
 - $V(G) \leq 10$: Well-structured, testable code.
 - $V(G) > 20$: High risk, complex, difficult to test.
 - $V(G) > 50$: Unmaintainable spaghetti code; refactoring is mandatory.

Concept Check Question 5: Advanced Coverage Criteria

In safety-critical avionics testing conforming to DO-178B/C Level A, why is MC/DC mandated rather than Multiple Condition Combination (MCC) or standard Branch Coverage?

- **A.** Because MC/DC is a black-box method that does not require examining boolean operators in code.
- **B.** Because MC/DC verifies condition independence with $N + 1$ tests, avoiding the 2^N explosion of MCC while being far stricter than Branch Coverage.
- **C.** Because MC/DC executes every loop iteration up to $N + 1$ times, guaranteeing zero runtime deadlocks.
- **D.** Because MC/DC eliminates the need for unit testing by proving requirement validity directly.

Concept Check Question 5: Answer & Explanation

- **Correct Answer: B**

- **Explanation:**

- DO-178B/C Level A mandates MC/DC because standard Branch Coverage is insufficient to verify that individual boolean conditions cannot cause hazardous hidden failures.
- While Multiple Condition Combination (MCC) checks all truth combinations, its exponential cost (2^N) is prohibitive for complex logic. MC/DC delivers rigorous condition independence verification in $N + 1$ **test cases**, providing maximum defect detection with linear efficiency.
- *Why others are incorrect:* Option A is false because MC/DC is an advanced white-box technique. Option C confuses MC/DC with loop boundary testing. Option D is false because MC/DC is a unit-level structural criterion and does not replace requirement validation.

7.6 Testing Synthesis & Anti-Patterns

"Beware of the fallacy: 100% code coverage does not mean 100% bug-free software." „

The Myth of 100% Code Coverage

- **1. Diminishing Return on Investment (ROI):**

- Moving a codebase from 0% to 80% coverage catches roughly 90% of all latent defects cost-effectively.
- Pushing from 95% to 100% coverage requires exponential engineering effort for minimal defect reduction.

- **2. Execution Does NOT Imply Correctness (The Assertion Void):**

- A test suite can achieve 100% Statement and Branch Coverage **without containing a single assertion statement!**
- If tests do not assert correct state, exceptions, and outputs, code execution reveals nothing about functional accuracy.

- **3. Missing Logic & Omitted Specifications:**

- Code coverage only measures code that was *written*.
- If a developer completely forgot to implement a critical business requirement (e.g., tax calculation for international orders), coverage metrics report 100% because no code exists to be missed!

- **4. Brittle Tests on Defensive Code:**

- Forcing coverage on hardware panic handlers, out-of-memory guards, and defensive logging produces brittle, low-value tests.

Comprehensive Comparison: Structural Coverage on Benchmark Code

Coverage Criterion	Target Evaluated	Benchmark Test Cases Needed	Key Strength / Practical Limitation
Statement (SC)	Executable statements	1 Test: (2, 0, 3)	Baseline requirement; blind to missing branches & logic defects.
Branch (BC)	Decision outcomes (T/F)	2 Tests: (3, 0, 3), (3, 1, 1)	Standard commercial baseline; guarantees 100% Statement Coverage.
Condition (CC)	Atomic conditions (T/F)	2 Tests: (2, 0, 3), (1, 1, 1)	Tests condition primitives; does NOT guarantee Branch Coverage!
MC/DC	Condition independence	3 Tests ($N + 1$): (2, 0), (1, 0), (2, 1)	DO-178B/C avionics standard; linear scaling with maximum rigor.
Multiple Cond (MCC)	Condition combinations	4 Tests (2^N): TT, TF, FT, FF	Exhaustive truth table testing; suffers from exponential explosion.
Path (PC)	Entry-to-exit paths	4 Tests	Complete path execution; suffers from catastrophic loop explosion.

Chapter Recap: Fill-in-the-Blank Challenge

Test your mastery of the core concepts across Chapter 7:

- **1.** The Pareto Principle in software testing states that roughly % of defects cluster in % of modules.
- **2.** Under the Single Fault Assumption, Independent Normal Boundary Value Analysis requires test cases for n variables.
- **3.** testing ensures that every pair of parameter values is tested together at least once, preventing combinatorial explosion.
- **4.** Achieving 100% Branch Coverage strictly guarantees 100% Coverage, but does not guarantee Condition Coverage.
- **5.** The safety-critical standard mandated for commercial flight software that achieves condition independence in $N + 1$ tests is .
- **6.** Basis Path Testing utilizes McCabe's Complexity to determine the minimum number of independent execution paths.

Chapter Recap: Solution Key

Here are the completed principles and technical terms:

- **1.** The Pareto Principle in software testing states that roughly **80%** of defects cluster in **20%** of modules.
- **2.** Under the Single Fault Assumption, Independent Normal Boundary Value Analysis requires $4n + 1$ test cases for n variables.
- **3. All-Pairs (Pairwise)** testing ensures that every pair of parameter values is tested together at least once, preventing combinatorial explosion.
- **4.** Achieving 100% Branch Coverage strictly guarantees 100% **Statement** Coverage ($BC100 \implies SC100$), but does not guarantee Condition Coverage.
- **5.** The safety-critical standard mandated for commercial flight software that achieves condition independence in $N + 1$ tests is **MC/DC**.
- **6.** Basis Path Testing utilizes McCabe's **Cyclomatic** Complexity to determine the minimum number of independent execution paths.

Chapter Summary & Engineering Best Practices

- **Synthesize Black-Box & White-Box Testing:**
 - Use **Equivalence Partitioning and Boundary Value Analysis** to design functional tests from specifications without implementation bias.
 - Use **Pairwise Testing** to tame multi-parameter configuration spaces.
 - Use **Branch and MC/DC Coverage** to verify that internal logic and safety conditions are thoroughly exercised.
- **Follow the Testing Pyramid:**
 - Anchor quality on a dense base of fast, automated unit tests.
 - Validate inter-service contracts with integration and component tests.
 - Reserve slow end-to-end tests for critical business user journeys.
- **Adopt Continuous Quality:**
 - Shift-left by reviewing requirements and architecture early.
 - Automate regression test suites within CI/CD pipelines on every pull request.
 - Remember Dijkstra's lesson: testing demonstrates defect presence—design code for testability from day one!